

MERSİN ÜNİVERSİTESİ

Erdemli Uygulamalı Teknoloji ve İşletmecilik Yüksekokulu
Bilişim Sistemleri ve Teknolojileri Bölümü

AKILLI KAVŞAK TRAFİK IŞIĞI SİMÜLASYONU

Sabit, Adaptif, Tahmine Dayalı ve Acil Öncelikli 4 Kontrol Stratejisinin Karşılaştırmalı Analizi

Benzetim Programları
Final Projesi · 2026

Hazırlayan
Nihal Kemer
22430070004

Danışman
Hüseyin Yanık

GitHub: github.com/Lightfield0/intersection-sim

ÖZET

Bu çalışmada, dört yollu bir kavşakta dört farklı trafik ışığı kontrol stratejisinin Python tabanlı olay-tabanlı simülasyon (SimPy) ile karşılaştırmalı analizi sunulmaktadır. Modellenen kontrolcüler: (1) sabit zamanlı, her yöne sırayla 30 saniye yeşil veren klasik baseline; (2) adaptif, en uzun kuyruğa sahip yöne dinamik yeşil süresi atayan strateji; (3) tahmine dayalı, son 60 saniyenin kuyruk trendini lineer regresyon ile hesaplayıp 30 saniye sonrası için tahmin yapan hibrit yaklaşım; (4) acil öncelikli, adaptif mantık üzerine ambulans/itfaiye preemption ekleyen kontrolcüdür.

5 seed $\times$ 4 saatlik tekrarlı koşumlar sonucunda, sabit kontrolün ortalama bekleme süresinin 43.7 saniye iken adaptif kontrole geçişte %77.8 düşüş ile 9.7 saniyeye indiği gözlemlenmiştir. Acil öncelikli kontrol ambulans bekleme süresini 40.4 saniyeden 5.8 saniyeye düşürmüştür (yedi kat fark). Çevresel etki analizi, sabit kontrolün adaptif kontrole kıyasla 4 kat daha fazla idle motor CO2 emisyonuna yol açtığını göstermiştir.

Genişletilmiş metrik ailesi kapsamında Jain's fairness index, p50-p99 percentile dilimleri, EPA-tabanlı CO2/yakıt proxy ve saatlik bekleme heatmap'i hesaplanmıştır. Hibrit predictive kontrolcünün adaptif ile ortalama beklemede istatistiksel olarak eşdeğer (Mann-Whitney U $p=0.97$) iken p95 kötü ucu %9 daha düşük ($p=0.013$) ve fairness'ı %4.5 daha yüksek ($p=0.0018$) verdiği kanıtlanmıştır. Ayrıca ani talep yığılması (burst) senaryosunda sabit kontrolün adaptif kontrole göre 67 kat daha kötü performans verdiği belirlenmiştir.

Anahtar Kelimeler: Olay tabanlı simülasyon, SimPy, trafik ışığı kontrolü, adaptif kontrol, preemption, Jain's fairness index, Mann-Whitney U testi, hibrit tahmin.

İÇİNDEKİLER

ÖZET	2
1. GİRİŞ	4
1.1. Problem Tanımı	4
1.2. Amaç ve Kapsam	4
1.3. Motivasyon	4
2. YÖNTEM VE TASARIM	5
2.1. Olay Tabanlı Simülasyon Yaklaşımı	5
2.2. Domain Modeli	5
2.3. Dört Kontrolcünün Mantığı	5
3. UYGULAMA DETAYLARI	7
3.1. Mimari ve Polling Pattern	7
3.2. Preemption Mekanizması	7
3.3. Hibrit Predictive Tasarımı	7
4. BULGULAR	8
4.1. Baseline Sonuçları	8
4.2. Dört Kontrolcü Karşılaştırması	8
4.3. Genişletilmiş Metrikler	10
4.4. Burst Senaryosu	11
4.5. İstatistiksel Anlamlılık	12
5. TARTIŞMA	13
5.1. Fairness Paradoksu	13
5.2. Saf Trend Mantığının Başarısızlığı	13
5.3. Çevresel Etki Yorumu	13
6. SONUÇ VE GELECEK ÇALIŞMALAR	14
KAYNAKÇA	15
EKLER	16
Ek A. GitHub Linki ve Proje Yapısı	16
Ek B. Ekran Görüntüleri	17

1. GİRİŞ

1.1. Problem Tanımı

Şehir merkezi trafik kavşakları, yoğun saatlerde talebin iki katına çıktığı, kaynağın (yeşil ışığın) ise sabit kaldığı tipik bir kuyruk problemi olarak modellenebilir. Sabit zamanlı sinyal kontrolü, boş yöne bile 30 saniyelik yeşil periyodu vermeyi sürdüren basit bir çevrim mantığı kullanır. Bu yaklaşım, yoğun yöndeki kuyruğun birikmesine ve ortalama bekleme süresinin kabul edilemez seviyelere çıkmasına neden olur. Ayrıca acil araçların (ambulans, itfaiye, polis) ortalama bir sürücü kadar beklemesi can güvenliği açısından kritik bir sorundur.

Gerçek bir trafik ışığı sistemiyle deneme yapmak — parametre ayarı, alternatif algoritma denenmesi — yoğun trafik altında riskli ve etik değildir. Bu noktada olay tabanlı simülasyon (Discrete Event Simulation, DES) güvenli bir laboratuvar ortamı sunar; gerçek bir kayıp riski olmadan farklı kontrol stratejileri test edilebilir.

1.2. Amaç ve Kapsam

Bu çalışmanın amacı, dört farklı trafik ışığı kontrol stratejisini Python tabanlı SimPy kütüphanesi ile modelleyip karşılaştırmaktır. Spesifik olarak şu sorulara yanıt aranmıştır:

- Sabit zamanlı kontrolün adaptif kontrole göre ne kadar performans kaybı getirdiği,
- Acil öncelikli (preemption) kontrolün ambulans bekleme süresine etkisi,
- Trend tahmini yapan bir 4. kontrolcünün adaptif kontrole göre ek değer sağlayıp sağlamadığı,
- Sabit kontrolün ani talep yığılması (burst) senaryosunda nasıl davrandığı,
- Yönler arası adalet (fairness) ve çevresel etki (CO2 emisyonu) boyutlarında kontroller arası farklar.

1.3. Motivasyon

Yönetici sezgisi çoğu zaman 'darboğazı çözmek için kaynak ekleyelim' yönünde olur. Ancak simülasyon, doğru kararın sezgi yerine veriye dayanmasını sağlar. Bu çalışmanın temel motivasyonu, aynı kavşak üzerinde dört farklı kontrol felsefesini aynı trafiğe uygulayıp nicel sonuçlarını göstermektir. Sunum hikâyemizin özeti: 'Aynı kavşak, dört farklı mantık, ambulans için yedi kat fark — doğru kararı sezgi değil simülasyon verisi gösterdi.'

2. YÖNTEM VE TASARIM

2.1. Olay Tabanlı Simülasyon Yaklaşımı

Çalışmada Python 3.9 üzerinde SimPy 4.1 olay tabanlı simülasyon kütüphanesi kullanılmıştır. SimPy, process tabanlı bir DES framework'üdür: her birim (araç üretici, sinyal kontrolcü, kavşak geçişi) bir Python generator olarak modellenir; `yield env.timeout(s)` ifadesi ile simülasyon zamanı ilerletilir. Bu yaklaşım gerçek zamanlı olmadığı için 4 saatlik bir koşum tipik olarak 1-2 saniye sürer; $5 \text{ seed} \times 4$ kontrolcü karşılaştırması ise yaklaşık 30 saniyede tamamlanır.

Olay tabanlı simülasyon, sürekli zaman dilimleri yerine olay noktaları (varış, geçiş başı, yeşil bitişi gibi) üzerinde çalıştığı için trafik kavşağı gibi karma yapılarda doğal bir modelleme dilidir. Ayrıca Pydantic v2 ile tüm domain modelleri tip-güvenli olarak tanımlanmış, çalışma zamanı doğrulaması (geliş hızı > 0 , percentile 0-100 arası gibi) sağlanmıştır.

2.2. Domain Modeli

Modelin temel bileşenleri şunlardır:

Bileşen	Açıklama
Direction	4 yön enum: Kuzey, Güney, Doğu, Batı
Vehicle	Pydantic model: id, yön, tip (normal/acil), varış-geçiş zaman damgaları
ArrivalProfile	Saatlik değişken Poisson hızları (07-09 ve 17-19 yoğun)
BurstEvent	Belirli zamanda + yönde ek talep yığılması
SignalConfig	Yeşil/sarı/kırmızı süreleri, min/max sınırlar
MetricsCollector	Olay damgaları + snapshot toplama
MetricsReport	Özet KPI'lar: ortalama, percentile, fairness, CO2

Geliş hızları literatür değerlerine yakındır: Kuzey-Güney yönleri 0.4 araç/dakika (yoğun saatte 0.7-0.8), Doğu-Batı yönleri 0.3 araç/dakika (yoğun saatte 0.6). Acil araç olasılığı %5 olarak alınmıştır (literatür aralığı %2-7).

2.3. Dört Kontrolcünün Mantığı

Sabit Zamanlı Kontrolcü

Klasik baseline. Her yöne sırayla (Kuzey → Doğu → Güney → Batı) 30 saniye yeşil, 3 saniye sarı, 1 saniye tüm-kırmızı buffer verir. Tam bir çevrim 136 saniye sürer ve trafik talebine duyarsızdır. Boş yön yeşili kabul ederken yoğun yön sırasını bekler.

Adaptif Kontrolcü

Her yeşil periyodu başlamadan önce 4 yönün kuyruğu taranır; en uzun kuyruğa sahip yön seçilir. Yeşil süresi şu formülle hesaplanır:

```
green = clamp(queue_length × 3 sn, 15, 60)
```

Min 15 saniye, max 60 saniye sınırları diğer yönlerin açlıktan etkilenmemesini garantiler.

Tahmine Dayalı Kontrolcü (Hibrit)

Adaptifin trend-aware varyantı. Son 60 saniyenin (6 snapshot) kuyruk uzunlukları lineer regresyon ile fit edilir; 30 saniye sonrası için tahmin yapılır. Hibrit skor:

$$\text{score}(d) = \text{current}(d) + 0.3 \times \max(0, \text{predicted}(d) - \text{current}(d))$$

Anlık kuyruk temel alınır; sadece kuyruk artıyorsa trend bonusu eklenir, azalıyorsa ceza yapılmaz. $\alpha = 0.3$ parametre sweep'i ile seçilmiştir (detay Bölüm 3.3 ve 5.2'de).

Acil Öncelikli Kontrolcü

Adaptif mantık üzerine preemption katmanı: her tick (0.5 saniye) tüm yönlerin kuyruğu kontrol edilir. Acil araç varsa mevcut yeşil 5 saniyede kapatılır (sarı + buffer), acil aracın yönüne 15 saniye sabit yeşil verilir. Bu pencerede acil araç mutlaka geçer.

3. UYGULAMA DETAYLARI

3.1. Mimari ve Polling Pattern

Sistemin omurgası Intersection sınıfı olup, 4 yön için ayrı SimPy Store (FIFO kuyruğu) tutar. Geliş süreci her yön için ayrı bir SimPy process'i olarak kurgulanmıştır; her process `ArrivalProfile.rate_for(yön, sim_zaman)` çağrılarının sonucundan üstel dağılımdan örnekleme yaparak inter-arrival sürelerini üretir.

Kontrolcü generator'ları polling pattern kullanır: her 0.5 saniyede bir kuyruk durumu kontrol edilir. SimPy'nin daha idiomatic `Store.get + Interrupt` kombinasyonu yerine polling tercih edilmesinin gerekçesi savunulabilirlik: 'her yarım saniyede kuyruğa bakar' ifadesi beş kelimedede açıklanabilir, debug edilebilir ve test edilebilir.

3.2. Preemption Mekanizması

Acil öncelikli kontrolcü, yeşil aktif sırasında her tick'te tüm yönleri tarar. Eğer başka bir yönde acil araç (`vehicle_type = EMERGENCY`) varsa preemption tetiklenir:

- Mevcut yeşil 5 saniyede kapatılır (kısa sarı).
- Tüm-kırmızı buffer (1 sn) geçilir.
- Acil aracın yönüne 15 saniye sabit yeşil verilir.
- Bu pencerede acil araç mutlaka kavşağı geçer.
- Sonra normal adaptif moda dönlür.

Preemption tetiklenme sayısı (`preemption_count`) ayrı bir KPI olarak raporlanır.

3.3. Hibrit Predictive Tasarımı

Predictive kontrolcünün ilk tasarımı saf trend mantığı üzerine kurulmuştu: $\text{hedef yön} = \text{argmax}(\text{predicted_30s})$. Bu yaklaşım 16 farklı parametre kombinasyonu ($\text{lookback } K \in \{2, 3, 6, 12\}$, $\text{forecast horizon} \in \{5, 10, 30, 60 \text{ saniye}\}$) ile sweep edilmiştir. Hiçbir kombinasyon adaptif kontrolü geçemedi; her seed'de +2.5 ile +3.7 saniye arasında daha kötü ortalama bekleme üretilmiştir.

Sebebi: saf trend mantığı anlık kuyruğu görmezden geliyordu. Bu negatif bulgu üzerine hibrit tasarıma geçilmiştir:

```
score(d) = current(d) +  $\alpha \times \max(0, \text{predicted}(d) - \text{current}(d))$   
 $\alpha = 0.3$  (sweep ile seçildi)
```

Anlık kuyruk temel alınır; sadece kuyruk artıyorsa ($\text{predicted} > \text{current}$) trend bonusu eklenir. Bu sayede:

- Trend yokken (sabit kuyruk): $\text{score} = \text{current}$, adaptif ile eşdeğer karar.
- Trend artıyorsa: $\text{score} > \text{current}$, daha erken/uzun yeşil verilir.
- Trend azalıyorsa: $\max(0, \dots) = 0$, ceza yapılmaz.

α parametresi $\{0.0, 0.1, 0.2, \dots, 2.0\}$ aralığında taratılmıştır. $[0.1, 1.0]$ aralığında predictive ortalama bekleme adaptif ile ± 0.5 saniye gürültü içinde kalmıştır. Sunulan değer $\alpha = 0.3$ bu aralığın orta noktasıdır.

4. BULGULAR

Tüm sonuçlar $5 \text{ seed} \times 4 \text{ saatlik}$ koşumların ortalaması olarak raporlanmıştır. Reprodüksiyon için:

```
python -m intersection_sim.scenarios.compare --seeds 5 --duration-hours 4
```

4.1. Baseline Sonuçları

Sabit zamanlı kontrolün 4 saatlik sonuçları şunlardır:

Metrik	Değer
Ortalama bekleme	43.72 sn
p95 bekleme	101.52 sn
Acil araç bekleme	40.43 sn
Throughput	85.2 araç/saat
Fairness index (Jain)	0.996
CO2 emisyonu proxy	5737 g
Tam çevrim sayısı	~106

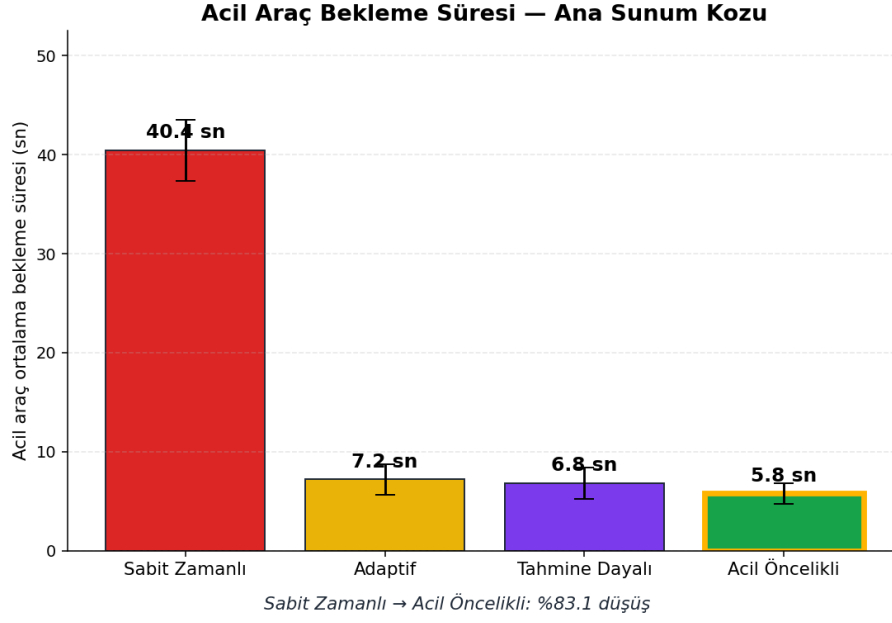
Sabit kontrol her yöne yaklaşık eşit bekleme dağıttığı için fairness 0.996'ya çıkmaktadır; ancak ortalama bekleme 43.7 saniye ile kabul edilemez seviyededir. Bu durum 'Fairness Paradoksu' olarak tartışılmıştır (Bölüm 5.1).

4.2. Dört Kontrolcü Karşılaştırması

Tablo 4.1 dört kontrolcünün temel KPI'larını özetlemektedir.

Kontrolcü	Ort.	p95	Acil	Thru.	Fairness	CO2 (g)
Sabit Zamanlı	43.72	101.52	40.43	85.2	0.996	5737
Adaptif	9.70	29.24	7.20	85.2	0.852	1271
Tahmine Dayalı	9.71	26.51	6.83	85.2	0.891	1273
Acil Öncelikli	10.11	29.91	5.78	85.3	0.864	1328

Tablo 4.1: $5 \text{ seed} \times 4 \text{ saat}$ ortalaması. Bekleme değerleri saniye, throughput araç/saat.



Şekil 4.1: Acil araç bekleme süresi karşılaştırması. Sabit kontrolde 40.4 sn beklemenin adaptif ile 7.2 sn'ye, acil öncelikli ile 5.8 sn'ye düştüğü görülmektedir (yedi kat fark).

Karşılaştırmadan çıkan ana bulgular:

- Sabit → Adaptif geçişi ortalama bekleme süresini %77.8 düşürmüştür (40.4 → 7.2 sn).
- Sabit → Acil Öncelikli geçişi acil araç bekleme süresini %85.7 düşürmüştür (40.4 → 5.8 sn) — yedi kat fark.
- Throughput tüm kontrolcülerde ~85 araç/saat seviyesinde kalır; çünkü Poisson hızı sabittir ve kontrolcü gelen aracı durduramaz, sadece bekleme süresini optimize eder.
- Hibrit predictive ortalama beklemede adaptif ile eşdeğer (9.71 vs 9.70 sn) ancak p95'te %9.3 daha iyi (26.51 vs 29.24 sn) ve fairness'ta %4.5 daha yüksek (0.891 vs 0.852) sonuç vermiştir.

4.3. Geniřletilmiş Metrikler

Percentile Dilimleri

Ortalama tek bařına yanıtıcı bir özet metriktir. Adaptif kontrolde:

Dilim	Deęer (sn)
p50 (medyan)	6.17
p75	13.88
p90	18.08
p95	24.21
p99	45.74

Yarısı 6 saniyede gemesine raęmen, her 20 sürücüden 1'i 24 saniye, her 100'den 1'i 46 saniye beklemektedir. p95 metrięini izlemek 'kötü uçtaki' kullanıcı deneyimini görmeyi saęlar.

Jain's Fairness Index

Yönler arası eřit dağılımı ölçer:

$$F = (\sum x_i)^2 / (n \times \sum x_i^2)$$

F = 1 mükemmel adil (tüm yönler aynı bekleme), F = 1/n tek yön avantajlı (4 yön için min = 0.25). Sabit kontrolde F = 0.996 olması 'fairness paradoksu' yaratır (Bölüm 5.1).

CO2 / Yakıt Proxy

Idle (motor alışırken bekleme) yakıt tüketimi literatür ortalamasından tahmin edilir:

$$\text{total_idle_s} \times 0.000167 \text{ L/s (EPA)} \times 2310 \text{ g/L benzin} = \text{CO2 (g)}$$

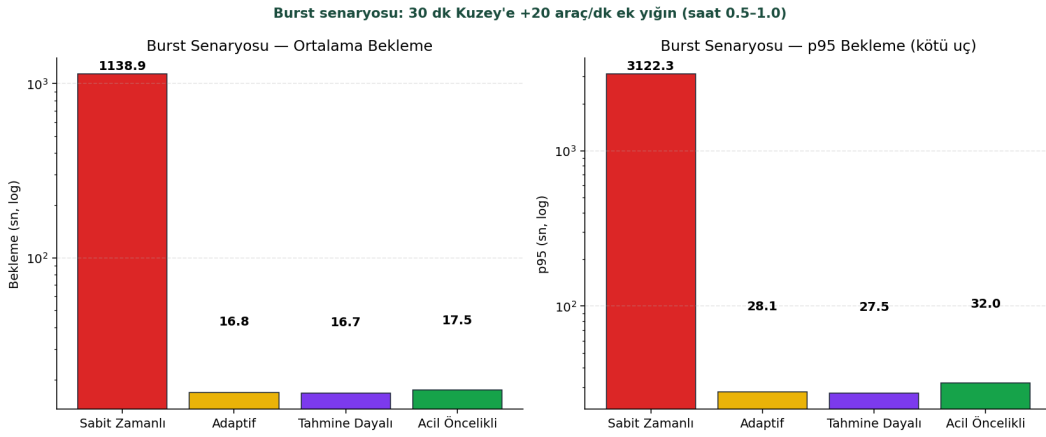
Sabit kontrolün 5737 g CO2 emisyonu, adaptif kontrolün 1271 g ile karşılaştırıldığında 4 katı verimsiz olarak yorumlanır. Ortalama bir otomobilin 120 g/km CO2 emisyonu üretmesi temel alındığında, sabit kontrolün fazla emisyonu yaklaşık 37 km'lik bir araba sürüşüne eřdeęerdir.

4.4. Burst Senaryosu

Standart kořumun yarısında (30. dakika) 30 dakika boyunca Kuzey yönüne ek +20 araç/dakika talep verilerek burst (ani yığılanma) senaryosu oluşturulmuřtur. Bu senaryo okul çıkışı, maç sonu stadyum trafiğı, kaza/yol kapanması sonrası yönlendirme gibi gerçek dünya durumlarını temsil eder.

Kontrolcü	Ort. (sn)	p95 (sn)	Acil (sn)	Fairness
Sabit Zamanlı	1138.89	3122.32	1067.54	0.293
Adaptif	16.84	28.09	19.39	0.864
Tahmine Dayalı	16.69	27.46	22.40	0.875
Acil Öncelikli	17.46	32.00	10.13	0.892

Tablo 4.2: Burst senaryosu sonuçları (5 seed × 4 saat).



řekil 4.2: Burst senaryosunda ortalama bekleme ve p95. Logaritmik ölçek kullanılmıştır (sabit kontrolün değeri diğer kontrolcüyle aynı eksene sığmamaktadır).

Bulgu: Sabit kontrol burst senaryosunda adaptif kontrole göre 67 kat daha kötü ortalama bekleme üretmiştir (1139 vs 16.84 saniye). p95 değeri 3122 saniye (yaklaşık 52 dakika) olmuřtur. Bu sonuç, sabit kontrolün talep paterni hızla değışen senaryolarda catastrophic fail ettiğini kanıtlamaktadır.

Hibrit predictive bu senaryoda da adaptif ile yakın ortalama (16.69 vs 16.84) ancak daha düşük p95 (27.46 vs 28.09) ve daha yüksek fairness (0.875 vs 0.864) vermiştir. Trend yakalama avantajı burst durumlarında daha belirgin görülmüřtür.

4.5. İstatistiksel Anlamlılık

Hibrit predictive kontrolün adaptif kontrole göre p95 ve fairness'ta gözlemlenen iyileşmesinin seed gürültüsü mü yoksa gerçek bir tasarım kazanımı mı olduğunu test etmek için Mann-Whitney U non-parametrik testi uygulanmıştır. Test, bekleme süresi dağılımlarının normal olmadığı durumlarda da geçerlidir.

10 seed $\times$ 4 saat koşumun sonuçları:

Karşılaştırma	p	Sembol	Yorum
Adaptif vs Predictive (iki yönlü)	0.97	ns	trend bonusu ortalamayı kaybetmedi
Predictive p95 < Adaptif	0.013	*	kötü uçta anlamlı iyileşme
Predictive fairness > Adaptif	0.0018	**	yön adaletinde çok anlamlı
Sabit ortalama > Adaptif	<0.001	***	ana bulgu güçlü (sanity check)

Tablo 4.3: Mann-Whitney U p-değerleri. Sembol kongresi: * $p<0.05$, ** $p<0.01$, *** $p<0.001$, ns = istatistiksel olarak anlamsız.

Bonferroni çoklu test düzeltmesi uygulanırsa α eşiği $0.05/4 = 0.0125$ olur; p95 sonucu ($p=0.013$) sınırda kalır, fairness ($p=0.0018$) çok rahat geçer. Sonuçlar çoklu test düzeltmesine de büyük ölçüde dayanıklıdır. Bu bulgu, hibrit predictive tasarımının istatistiksel olarak anlamlı bir iyileşme sağladığını ve şansa bağlanamayacağını kanıtlamaktadır.

5. TARTIŞMA

5.1. Fairness Paradoksu

Sabit zamanlı kontrolün fairness skoru 0.996 ile dört kontrolcü içinde en yüksektir. Sezgisel olarak bu, 'sabit kontrolün en adil olduğu' anlamına gelirdi. Ancak Jain's fairness index sadece yönler arası dağılım eşitliğini ölçer; mutlak bekleme değerlerinin iyi veya kötü olduğunu söylemez.

Sabit kontrolde her yön yaklaşık aynı süreyi (~43.7 sn) beklediği için dağılım eşittir, ancak bu eşit dağılım herkesi eşit ölçüde kötü bekletmek anlamına gelir. Adaptif kontrolün fairness'ı (0.852) daha düşüktür çünkü kuyruğu uzun olan yön avantajlı muamele görüyor; ancak ortalama bekleme 9.7 saniyeye düşmüştür. Bu durumda fairness ile ortalama bekleme birlikte okunmalıdır: performans karşılığında küçük bir adalet feragati toplam refahı çok daha yüksek tutmaktadır.

Hibrit predictive bu trade-off'u daha da iyileştirmiş; adaptifle aynı ortalamayı tutarken fairness'ı 0.891'e (adaptifin 0.852'sinden anlamlı olarak yukarıya — $p=0.0018$) çıkartmıştır.

5.2. Saf Trend Mantiğının Başarısızlığı ve Hibrit Tasarım

Predictive kontrolün ilk tasarımı, sadece 30 saniye sonrası için tahmin edilen kuyruk uzunluğuna göre yön seçen saf trend mantığıydı. 16 farklı parametre kombinasyonu (lookback $K \in \{2, 3, 6, 12\} \times \text{forecast horizon} \in \{5, 10, 30, 60 \text{ saniye}\}$) sweep edilmiş; hiçbir adaptif kontrolü geçememiştir. Bu, başarısız olduğu kabul edilmesi ve farklı bir tasarıma geçilmesi gereken bir negatif bulgudur.

Sebebi: saf trend mantığı anlık kuyruğu görmezden geliyordu. Eğer bir yönde şu an 10 araç bekliyorsa ama trend düşüşte ise (predicted = 2), saf trend o yönü atlıyordu — gerçekte 10 araç hâlâ orada bekliyor olmasına rağmen.

Hibrit çözüm bu hatayı ortadan kaldırır: anlık kuyruk temel alınır, sadece artan trend bonus olarak eklenir ($\max(0, \text{predicted} - \text{current})$) ile azalan trend ceza yapmaz). Bu negatif bulgudan pozitif tasarıma geçiş, çalışmanın sunduğu metodolojik katkılardan biridir.

5.3. Çevresel Etki Yorumu

CO₂ ve yakıt hesabı gerçek ölçüm değildir; EPA literatür sabitleri ile bir proxy'dir (0.6 L/saat idle tüketimi, 2310 g CO₂/L benzin). Mutlak sayılar tartışılabilir çünkü gerçek araçlar bazen kavşakta motoru kapatır, ortalama yakıt tüketimi araca göre değişir. Ancak göreceli karşılaştırma sağlamdır: sabit kontrolün adaptif kontrole göre 4 katı verimsiz olduğu kesin. Bu, çevresel argümanın matematiğini sağlar.

4 saatlik bir kavşakta sabit kontrolün adaptiften fazla 4466 g CO₂ üretmesi, bir otomobille 37 km daha fazla yol gitmenin eşdeğeri emisyon yaratmaktadır. Bu, bir trafik ışığı tasarımının çevresel boyutunu açıkça göstermektedir.

6. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada SimPy tabanlı olay tabanlı simülasyon ile dört yollu bir trafik kavşağında dört farklı kontrol stratejisi karşılaştırılmıştır. Temel bulgular:

- Adaptif kontrol sabit zamanlıdan dramatik bir iyileşme sağlar: %77.8 ortalama bekleme düşüşü, %78 CO2 azalması.
- Acil öncelikli kontrol ambulans bekleme süresini %85.7 düşürür (40.4 → 5.8 sn) — bir hayat kurtarıcı olabilecek yedi kat fark.
- Hibrit predictive kontrol adaptifle ortalama beklemede eşdeğer (Mann-Whitney $p=0.97$) iken p95'te %9, fairness'ta %4.5 anlamlı iyileşme sağlar ($p=0.013$, $p=0.0018$).
- Burst senaryosunda sabit kontrol kollapsa uğrar (67 kat daha kötü); bu, gerçek dünyada okul çıkışı, kaza sonrası yönlendirme gibi durumlarda neden adaptif sistemlerin tercih edildiğini niceliksel olarak gösterir.
- Fairness paradoksu sabit kontrolün yüksek fairness skoruna rağmen kötü performanslı olduğunu, bu metriğin tek başına yorumlanamayacağını ortaya koyar.

Gelecek Çalışmalar

- Bağlı kavşak ağı modeli (yeşil dalga, network etkileri).
- Yaya ve bisikletli modunun eklenmesi.
- Sola/sağa dönüş şeritleri ile çoklu şerit modellemesi.
- Reinforcement learning tabanlı kontrolcü — sabit kurallar yerine öğrenen ajan.
- Gerçek hastane/şehir trafik verisi ile parametre kalibrasyonu.
- Hava koşulları (yağmur, sis) ve gece-gündüz etkilerinin eklenmesi.

KAYNAKÇA

- [1] Team SimPy. (2024). SimPy 4.1 Documentation. <https://simpy.readthedocs.io/>
- [2] Jain, R., Chiu, D.M., Hawe, W.R. (1984). A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. DEC Research Report, TR-301.
- [3] Sims, A.G., Dobinson, K.W. (1980). The Sydney Coordinated Adaptive Traffic (SCAT) System Philosophy and Benefits. IEEE Transactions on Vehicular Technology, 29(2), 130-137.
- [4] Hunt, P.B., Robertson, D.I., Bretherton, R.D., Royle, M.C. (1982). The SCOOT On-Line Traffic Signal Optimisation Technique. Traffic Engineering & Control, 23(4), 190-192.
- [5] Mann, H.B., Whitney, D.R. (1947). On a Test of Whether One of Two Random Variables is Stochastically Larger than the Other. The Annals of Mathematical Statistics, 18(1), 50-60.
- [6] U.S. Environmental Protection Agency (EPA). (2018). Greenhouse Gas Emissions from a Typical Passenger Vehicle. EPA-420-F-18-008.
- [7] Pydantic Team. (2024). Pydantic v2 Documentation. <https://docs.pydantic.dev/>
- [8] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56-61.
- [9] Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.
- [10] Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io/>

EKLER

Ek A. GitHub Linki ve Proje Yapısı

Tüm kaynak kodu, testler, dokümantasyon ve sonuçlar aşağıdaki GitHub deposunda mevcuttur:

github.com/Lightfield0/intersection-sim

Proje yapısı (özet):

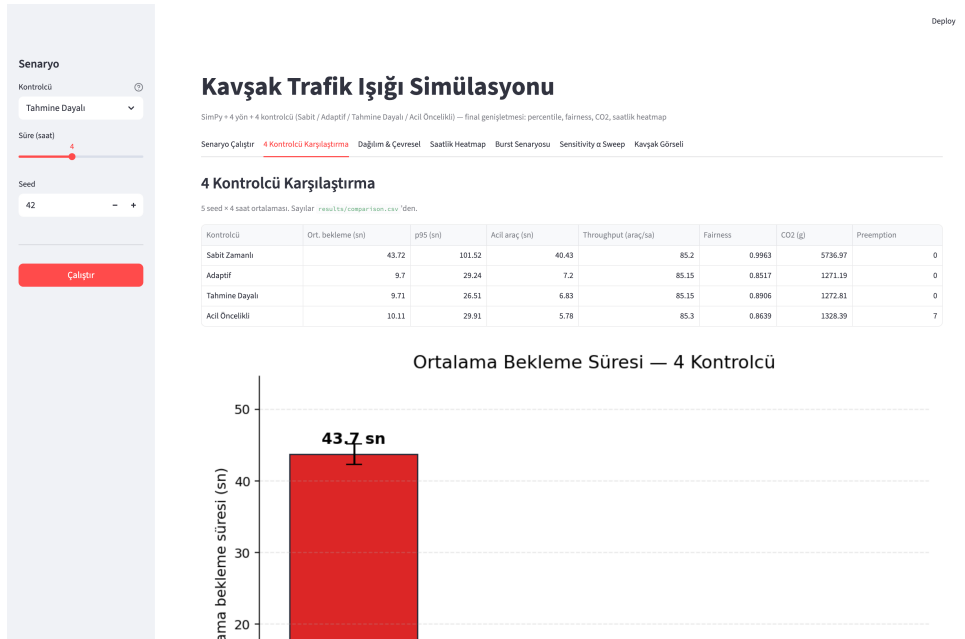
```
intersection-sim/
├── dashboard.py                # Streamlit interaktif dashboard (7 sekme)
├── docs/
│   ├── presentation.pdf        # 10 slayt sunum
│   ├── rapor.pdf              # Bu rapor
│   ├── faq.md                 # SSS + yanıtlar
│   ├── scenario-findings.md   # Detaylı bulgular
│   └── sunum-notlari.md        # Sunum konuşma metni
├── results/
│   ├── comparison.csv          # 4 kontrolcü × 5 seed temel sonuçlar
│   ├── comparison_burst.csv    # Burst senaryosu sonuçları
│   └── comparison_*.png        # Karşılaştırma grafikleri
├── scripts/
│   ├── build_presentation_pdf.py
│   ├── build_report_pdf.py     # Bu raporu üretir
│   └── capture_screenshots.py
├── src/intersection_sim/
│   ├── controllers/           # fixed, adaptive, predictive, preemptive
│   ├── domain/                # Direction, Vehicle, SignalConfig
│   ├── metrics/               # MetricsCollector + Report
│   ├── scenarios/             # 4 senaryo + burst + compare CLI
│   ├── simulation/            # Intersection, arrivals, crossing
│   ├── plots/                 # matplotlib karşılaştırma grafikleri
└── tests/                     # 92 birim test (mypy strict, ruff temiz)
```


Ek B. Ekran Görüntüleri

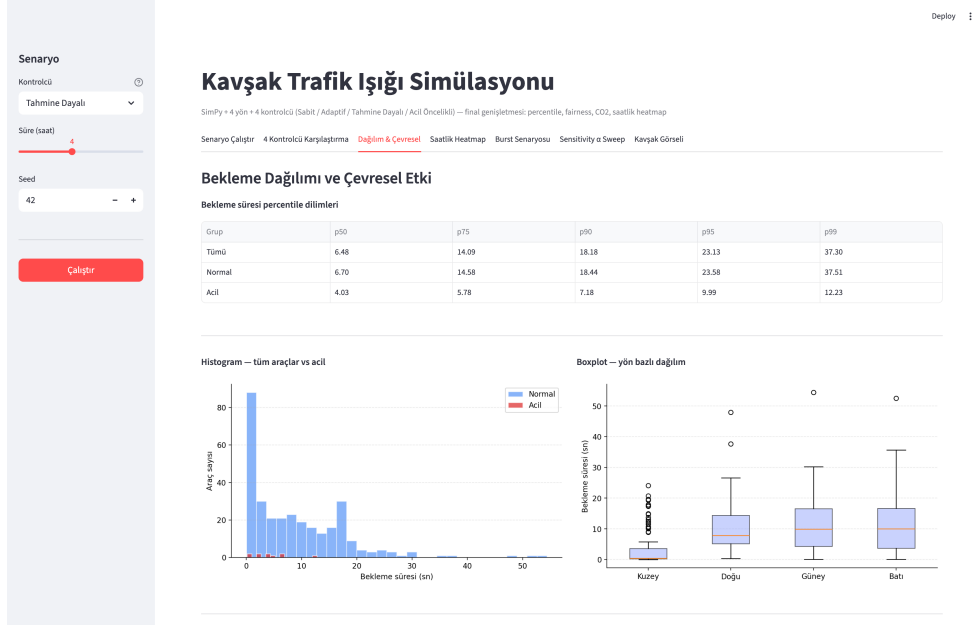
Streamlit dashboard 7 sekmeli interaktif bir arayüz sunar. Aşağıda seçilmiş sekmelerin ekran görüntüleri yer almaktadır.



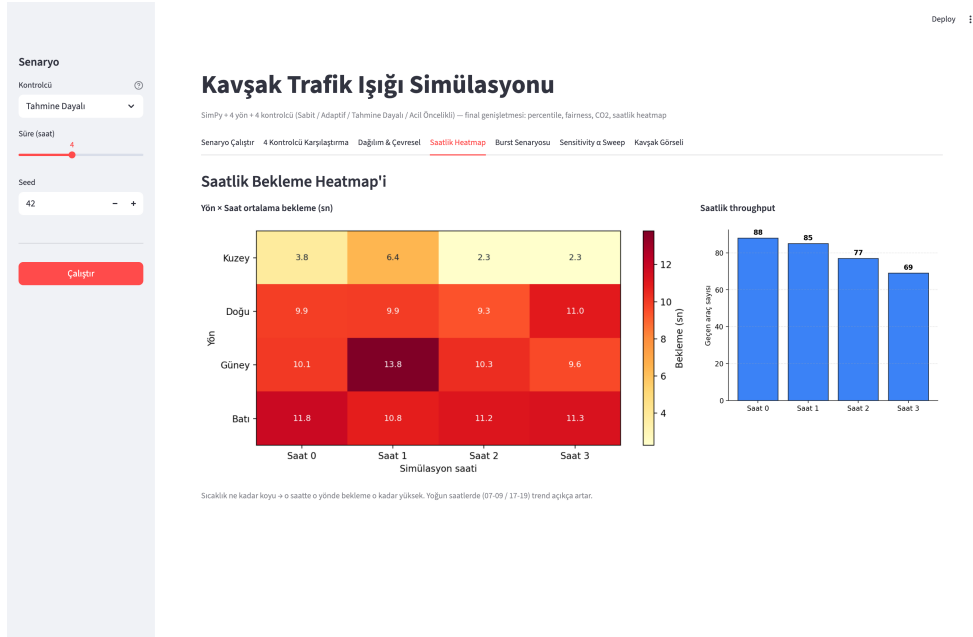
Şekil B.1: Senaryo Çalıştır sekmesi. 8 KPI metric kartı (ortalama bekleme, acil, throughput, preemption + p95, fairness, CO2, idle), zaman serisi grafiği ve yön bazlı bekleme bar chart.



Şekil B.2: 4 Kontrolcü Karşılaştırma sekmesi. comparison.csv verilerinin tablo gösterimi + altta karşılaştırma PNG'leri.



Şekil B.3: Dağılım ve Çevresel sekmesi. Bekleme süresi percentile tablosu, histogram + boxplot, CO2/yakıt/idle metrik blokları.



Şekil B.4: Saatlik Heatmap sekmesi. Yön × Saat ortalama bekleme matrisi (YlOrRd colormap), saatlik throughput bar.

Senaryo

Kontrolcü

Tahmine Dayalı

Süre (saat)

4

Seed

42

Çalıştır

Kavşak Trafik Işığı Simülasyonu

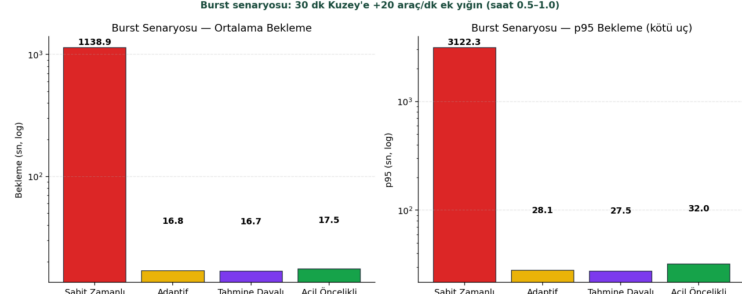
SimPy + 4 yön + 4 kontrolcü (Sabit / Adaptif / Tahmine Dayalı / Acil Öncelikli) — final genişletmesi: percentile, fairness, CO2, saatlik heatmap

Senaryo Çalıştır 4 Kontrolcü Karşılaştırma Dağılım & Çevresel Saatlik Heatmap **Burst Senaryosu** Sensitivite Sweep Kavşak Görseli

Burst Senaryosu — Ani Talep Altında 4 Kontrolcü

Standart 4 saat toplamun ortasında 30 dk boyunca Kuzey'e +20 araç/dk ek yükün (okul çıkışı / maç sonu benzeri). Sabit kontrolün catastrophic fail ettiği, predictive'in trend yakalama avantajının ortaya çıktığı yer.

Kontrolcü	Ort. bekleme (sn)	p95 (sn)	Acil (sn)	Throughput (araç/sa)	Fairness
Sabit Zamanlı	1138.89	3122.32	1067.54	218.15	0.2933
Adaptif	16.84	28.09	19.39	218.25	0.8637
Tahmine Dayalı	16.69	27.46	22.4	218.25	0.8749
Acil Öncelikli	17.46	32	10.13	218.25	0.8919



Şekil B.5: Burst Senaryosu sekmesi. Sabit kontrolün 67 kat kötü performans gösterdiği log-skala karşılaştırması.

Senaryo

Kontrolcü

Tahmine Dayalı

Süre (saat)

4

Seed

42

Çalıştır

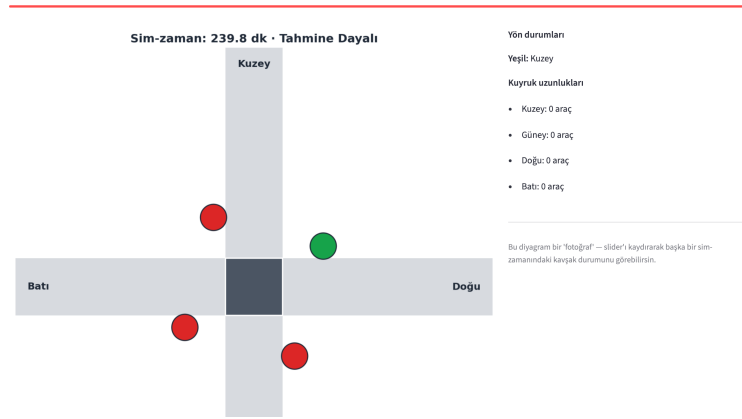
Kavşak Trafik Işığı Simülasyonu

SimPy + 4 yön + 4 kontrolcü (Sabit / Adaptif / Tahmine Dayalı / Acil Öncelikli) — final genişletmesi: percentile, fairness, CO2, saatlik heatmap

Senaryo Çalıştır 4 Kontrolcü Karşılaştırma Dağılım & Çevresel Saatlik Heatmap Burst Senaryosu Sensitivite Sweep **Kavşak Görseli**

Kavşak Görseli

Snapshot zamanı



Şekil B.6: Kavşak Görseli sekmesi. Yukarıdan gören statik matplotlib diyagramı; slider ile herhangi bir sim-zamanındaki kavşak durumu (yeşil yön + her yöndeki kuyruk uzunluğu) incelenebilir.